A Overview

- Appendix B shows the details of the benchmark task set.
- Appendix C lists the details to reproduce our results.
- Appendix D includes additional experiment results.
- Appendix E includes additional implementation details.
- Appendix F provides details on our real-world evaluation setup.

B Tasks

B.1 Simulation Tasks

We choose the five tasks from Robosuite [3, 31] (Fig. 5). We use the largest initiation range version (D2) for all tasks. The only exception is *Nut Assembly*, where we reduce the x-range of the nuts' placement to $(-0.15, 0.15)$ since the original range produces unreachable initial positions.

Observations. The observation consists of two (or three, in Threading) 84×84 RGB images and robot proprioception state, including a 6-dim end-effector pose composed of 3-dim Euclidean position in meters and 3-dim rotation represented in axis-angle form; a 2-dim gripper position, representing the distances of the two grippers to the center; and a 7-dimensional joint configuration. The front-view camera is demonstrated in Figure 5; the wrist camera angle is shown in Figure 4b (right); the extra Threading side-view camera is shown in Figure 4a (right).

B.2 Real-World Tasks

Square (first two stages of *Nut Assembly*) and *Threading* are selected for real-world evaluations (Fig. 6). We remove the legs of the ring stand in *Threading* for 3D-printing compatibility. We also use a limited initialization range due to the size and controllability of the robot arm. The specific ranges are:

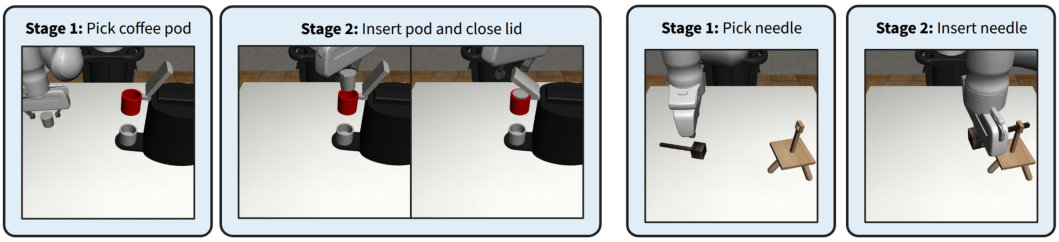
Square

- Square nut: $x \in (-0.05, 0)$, $y \in (0.05, 0.1)$, $z_{\text{rot}} \in (-30^\circ, 30^\circ)$
- Square peg: $x \in (-0.05, 0)$, $y \in (-0.1, -0.05)$, $z_{\text{rot}} \in (-30^\circ, 30^\circ)$

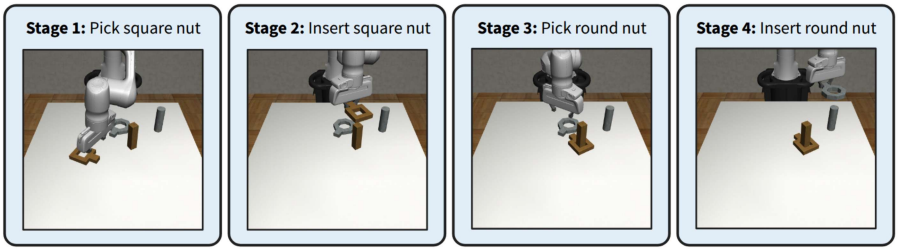
Threading

- Needle: $x \in (-0.05, 0)$, $y \in (0.05, 0.1)$, $z_{\text{rot}} \in (-30^\circ, 30^\circ)$
- Ring stand: $x \in (-0.025, 0.025)$, $y \in (-0.1, -0.05)$, $z_{\text{rot}} \in (-15^\circ, 15^\circ)$

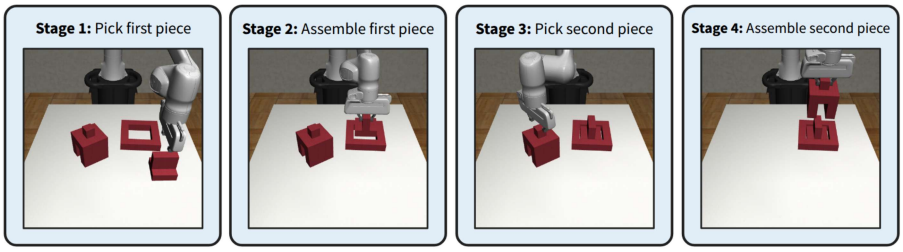
Observations. For simulation training, we use the same observation space as our simulation task setup. We do not include the third side-viewing camera in *Threading*, as the original two cameras suffice in this initialization setting. The robot joint configuration input also changes to 6 dimensions to accommodate the 6-DOF arm used in our evaluation.



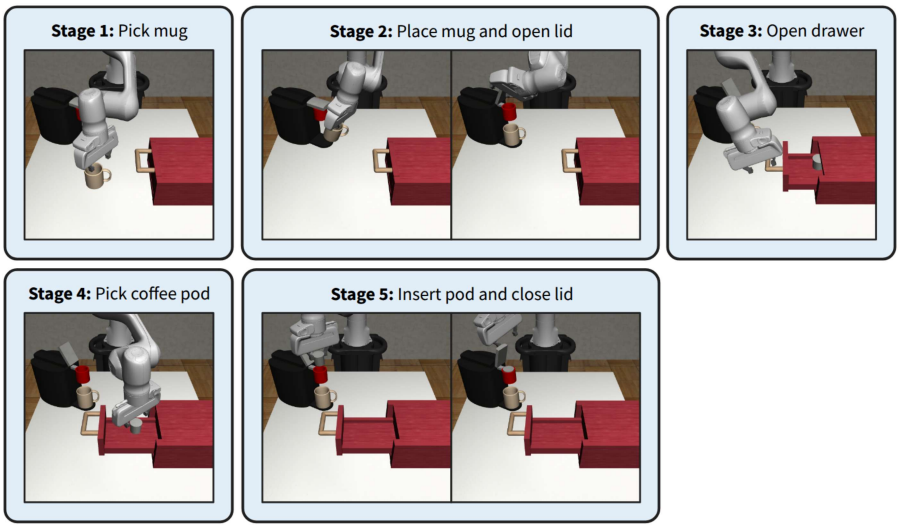
(a) Coffee (2 stages) **(b)** Threading (2 stages)



(c) Nut Assembly (4 stages)

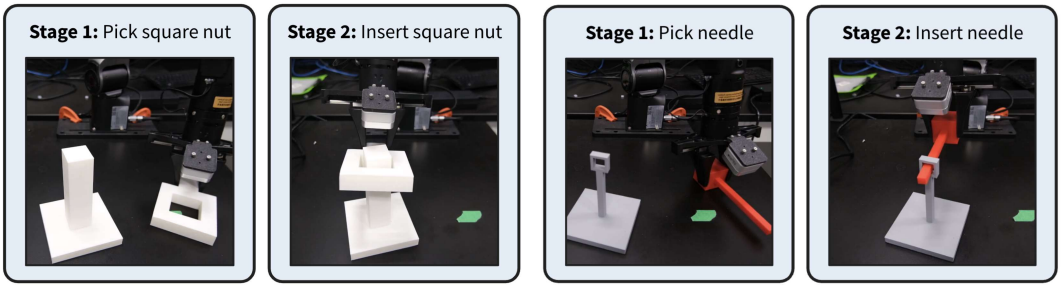


(d) Three Piece Assembly (4 stages)



(e) Coffee Preparation (5 stages)

Figure 5: All five simulation tasks showcased.



(a) Square (2 stages) **(b)** Threading (2 stages)

Figure 6: Two real-world tasks.

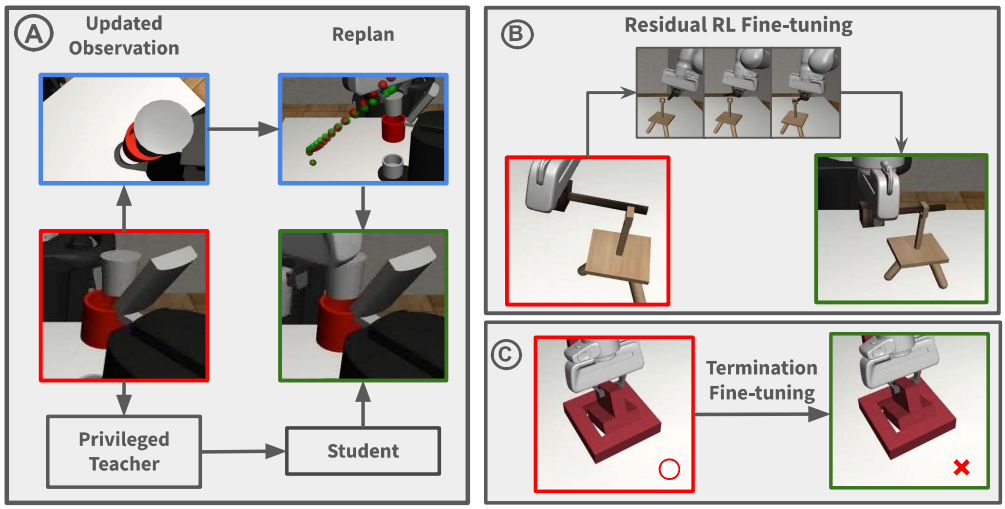


Figure 7: Depiction of how we fine-tune the three components. (A) The pose predictor based on image and robot state input is fine-tuned towards a teacher predictor with privileged object state information. During execution, it constantly updates its prediction when gathering new information about the target object and reroutes dynamically. (B) The skill policy is fine-tuned through residual reinforcement learning. (C) We purge the false-positive stage termination predictions that lower the completion rate of later stages.

C Reproducibility

C.1 Pseudocode

Algorithm 1 ReinforceGen Deployment Pseudocode

procedure REINFORCEGEN
  $o \leftarrow$ `env.reset()` ▷ Get initial observation
  for $i := 1 \rightarrow n$ **do**
    $\langle R_i, \mathcal{I}_{\theta_i}, \pi_{\theta_i}, \mathcal{T}_{\theta_i} \rangle \leftarrow \psi_{\theta_i}$ ▷ Skill ψ_i
    $p \leftarrow \mathcal{I}_{\theta_i}(o)$ ▷ Predict initiation pose
    $\tau \leftarrow$ `planToPose(env,` p`)` ▷ Motion planning
    for $a \in \tau$ **do**
      $o \leftarrow$ `env.step(`a`)` ▷ Execute motion action
      $p' \leftarrow \mathcal{I}_{\theta_i}(o)$
      if $\text{dis}(p, p') > \epsilon$ **then** ▷ Refined initiation prediction
        $p \leftarrow p'$
        $\tau \leftarrow$ `planToPose(env,` p`)` ▷ Replan trajectory
    while $\mathcal{T}_{\theta_i}(o) \neq 1$ **do** ▷ Until subgoal success
      $a \leftarrow \pi_{\theta_i}(o)$
      $o \leftarrow$ `env.step(`a`)` ▷ Execute policy action

C.2 Depiction of Fine-Tuning

Fig. 7 illustrates the fine-tuning process of ReinforceGen on the three components of an HSP.

C.3 Thresholds Used in ReinforceGen

Threshold for motion planner replanning (Sec. 4.1) We replan the motion planning trajectory when the pose distance (c.f. App. E.2) between the newest predicted pose and the current pose target exceeds **0.05**.

Threshold for termination rejection (Sec. 4.3) We reject terminations with a predicted task completion rate lower than **0.4**.

C.4 Hyperparameters for Data Generation and Imitation Learning

We follow the exact setup in [3].

C.5 Hyperparameters for Reinforcement Learning

Table 5: DrQ-v2 hyperparameters.

Network structure	CNN
Learning rate	1e-4
Discount	0.99
Batch size	256
n-step returns	3
Action repeat	1
Seed frames	4000
Feature dim	50
Hidden dim	1024
Optimizer	Adam
Training steps	2M
Constraint coef. (α)	5.0

We use DrQ-v2 [33] for skill fine-tuning (Sec. 4.2). The hyperparameters are shown in Tab. 5.

C.6 Usage of Fine-tuning Methods

Table 6: Usage of different fine-tuning methods in ReinforceGen

Task	Stage	Pose distillation (Sec. 4.1)	Skill fine-tune (Sec. 4.2)	Termination fine-tune (Sec. 4.3)
Coffee	1	✗	✗	✗
	2	✗	✓	✗
Threading	1	✗	✗	✗
	2	✗	✓	✗
Nut Assembly	1	✓	✗	✗
	2	✗	✓	✗
	3	✓	✓	✗
	4	✗	✗	✗
Three Piece	1	✗	✗	✓
	2	✗	✓	✓
	3	✗	✗	✓
	4	✗	✓	✓
Coffee Prep.	1	✗	✗	✗
	2	✗	✓	✗
	3	✗	✓	✗
	4	✗	✓	✗
	5	✗	✓	✗

By default, all stages in all tasks use first-iteration pose distillation and real-time replanning (Sec. 4.1). The usage of second-iteration pose distillation and skill/termination fine-tuning is listed in Tab. 6.

C.7 End-to-End Distillation

We add Gaussian noise $\mathcal{N}(0, \sigma^2)$ with $\sigma = 0.01$ during trajectory generation. For each task, we generate 3000 successful trajectories. We use the same hyperparameters as skill imitation for IL.

D Additional Experiments

D.1 Case Study on the Effectiveness of Replanning in *Nut Assembly*

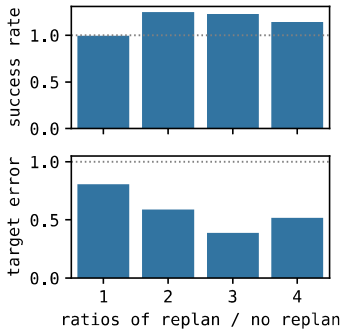


Figure 8: Ablation of real-time replanning at every stage of *Nut Assembly*. The top figure shows the per-stage success rate, and the bottom shows the pose target error. The numbers are ratios of applying replanning versus not applying.

To illustrate the effectiveness of real-time replanning (Sec. 4.1), we plot the reduction in prediction error and improvement in task success rate with replanning in all 4 stages in *Nut Assembly*. The results in Fig. 8 show that replanning significantly reduces the target prediction error in all stages, and in turn, improves the subsequent skill success rates.

D.2 Learned Termination Predictor

Learned termination predictors have limited impact on task completion. We evaluate ReinforceGen with a learned termination predictor that determines termination with partial observations. As shown in Table 7, using a learned termination predictor only introduces minor drops in success rates in *Threading* and *Three Piece*, while maintaining its performance in the other tasks.

Success Rate (%)	*Nut Assembly*	*Threading*	*Three Piece*	*Coffee*
Oracle Termination	85.80	82.20	80.40	93.81
Learned Termination	84.60	79.20	73.80	92.60

Table 7: ReinforceGen's learned termination predictor, which unlike the Oracle Termination does not have access to the state, performs well albeit slightly worse when compared to the Oracle Termination upper bound.

D.3 Ablation on Distillation

In Fig. 9, we add artificial noise to the actions and plot the relative performance decreases against the noise scale. In all tasks, ReinforceGen appears to be more resistant to action noise and trajectory deviations, matching our assertions in Sec. 4.5.

D.4 Real-World Evaluation on *Threading*

We also evaluated ReinforceGen on a real-world version of *Threading* (see Sec. B.2). We collected only 1 tele-op demonstration and generated 100 demonstrations for imitation learning to test the limit of ReinforceGen. We also only fine-tuned the second stage skill policy since it is the most challenging part of the task. The results are shown in Tab. 8.

Success Rate (%)	*Simulation Success.*	*Real Success.*	*Sim-to-Real Success.*
HSP	20.00	5.00	25.00
ReinforceGen	**75.00**	**35.00**	**46.67**

Table 8: Real-world evaluation results on *Threading*. Success rates are averaged over 20 rollouts. Sim-to-real transfer success rates are calculated by dividing real-world success rate by simulation success rate.

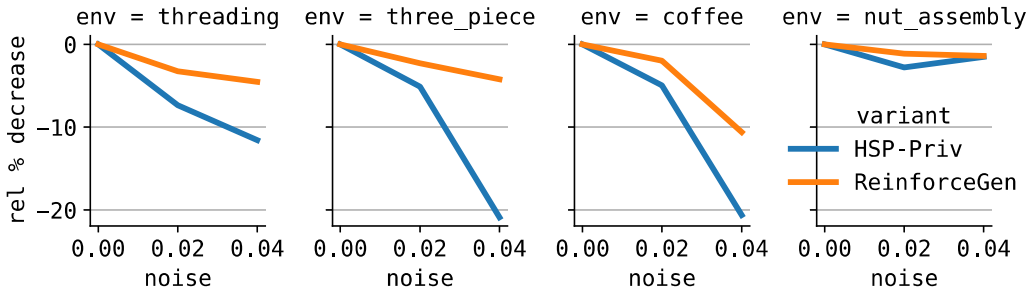


Figure 9: Comparing the resistance to action noise between HSP-Priv and ReinforceGen. For most tasks, ReinforceGen has a much higher tolerance to action noise.

Consistent with the conclusion drawn in the *Square* experiments, The ReinforceGen agent not only shows a significantly higher success rate than HSP on a real robot, but also is more effective in handling the sim-to-real gap.

E Additional Details

E.1 Pose Noise in Section 4.1

In Figure 3, we add artificial noise to initiation poses to demonstrate their relationship with skill completion rates. The noise is added with the following procedure. Let the original pose be $\boldsymbol{P} := \boldsymbol{p} \oplus \boldsymbol{r}$, where $\boldsymbol{p}$ is a 3-dimensional position vector in meters, $\boldsymbol{r}$ is a 3-dimensional rotation vector in the axis-angle form. A noisy version of $\boldsymbol{P}$ with a scale σ is defined as:

$$\tilde{\boldsymbol{P}} \leftarrow \boldsymbol{P} + \mathcal{N}(\mathbf{0}, \sigma^2 \boldsymbol{I}_6). \tag{1}$$

E.2 Pose Distance Metric

We use the following metric to compute the distance between two poses throughout our implementation. Let $\mathbf{P_1} := (\mathbf{p_1}, \mathbf{q_1})$, $\mathbf{P_2} := (\mathbf{p_2}, \mathbf{q_2})$ be the two poses to compare, $\mathbf{p_1}, \mathbf{p_2}$ the Euclidean positions, and $\mathbf{q_1}, \mathbf{q_2}$ the quaternion-form rotations.

$$d^{\text{pos}}(\mathbf{P_1}, \mathbf{P_2}) := \sqrt{(\mathbf{p_1} - \mathbf{p_2})^\top (\mathbf{p_1} - \mathbf{p_2})} \tag{2}$$

$$d^{\text{rot}}(\mathbf{P_1}, \mathbf{P_2}) := \frac{1}{\pi} \min\{\cos^{-1}(\mathbf{q_1}^\top \mathbf{q_2}), \cos^{-1}(-\mathbf{q_1}^\top \mathbf{q_2})\} \tag{3}$$

$$d^{\text{pose}}(\mathbf{P_1}, \mathbf{P_2}) := d^{\text{pos}}(\mathbf{P_1}, \mathbf{P_2}) + d^{\text{rot}}(\mathbf{P_1}, \mathbf{P_2}) \tag{4}$$

F Real-World Setup

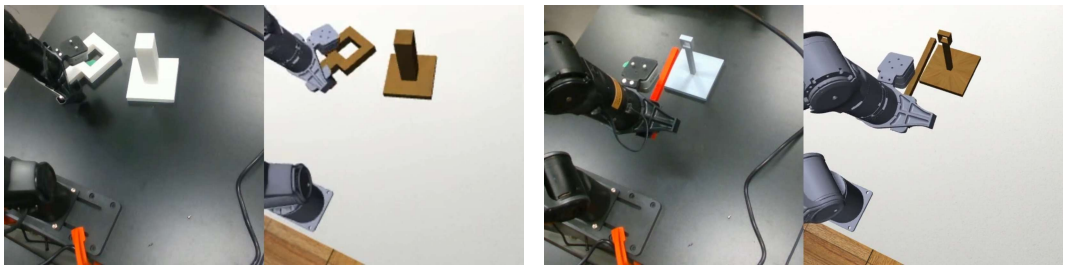
We provide additional details on our real-world evaluations. See Sec. B.2 for the details of the real-world tasks. See Sec. D.4 for evaluation results on *Threading*.

Robot. We use an AgileX PiPER 6-DOF arm for all our real-world evaluations. The arm is mounted at a 90-degree angle, facing the negative y-axis.

Pose estimation. We use a fixed RealSense D435i camera to capture the scene and FoundationPose [32] to estimate the object poses. We do pose estimation once at the beginning of an episode.

Demonstrations. We use another PiPER teacher arm to tele-op the main arm and collect demonstrations in the real world. We then replay the joint position trajectory of the tele-op demonstrations in a MuJoCo simulator and convert them into a simulation-based demonstration dataset. For *Square*, we collected 10 tele-op demonstrations and generated a 1,000-demonstration dataset. For *Threading*, we collected 1 tele-op demonstration and generated a 100-demonstration dataset. We use the same setup as our simulation evaluations to train a BC policy.

Fine-tuning. Since pose estimation is required in our workflow, we do not need to do object-state-agnostic initiation pose prediction. We therefore use the privileged pose predictor as in HSP-Priv and omit the initiation pose prediction fine-tuning process. We use the same setup as in our simulation experiments for skill and termination fine-tuning. For *Threading*, we omit termination fine-tuning and only fine-tune the second-stage skill policy.



(a) Square **(b)** Threading

Figure 10: Sim-to-real deployment.

Deployment. We use the following procedure to deploy our simulation-based policy on a real robot. First, we use FoundationPose to estimate the poses of the objects in the real world. We then sync the estimated poses in our MuJoCo simulation and run the evaluated policy in simulation, while recording the joint position and gripper action trajectory. If the simulation is successful, we finally replay the trajectory in the real world by sending joint and gripper commands to the robot arm. See Fig. 10 for an illustration.